MODULE-BASED PROJECT DOCUMENT

SkillLens

Module-Based Project Document

Project	SkillLens — Skill-to-Employment Intelligence Platform
Institute	Swami Vivekanand College of Engineering, Indore
Mentor	Ms. Amisha Jain
Authors	Teena Devray · Rohit Solanki · Vishal Panwar
Date	March 2026

1. Purpose

This document describes every module of SkillLens from a technical and architectural perspective — how each module is internally structured, what it depends on, what data it reads and writes, how it connects to other modules, and what specific logic drives its core behaviour. It is the developer's reference document, complementing the feature document (which is user-centric) and the SRS (which is requirement-centric).

2. System Architecture Overview

SkillLens follows a standard three-tier MERN architecture with one external service integration:

Tier	Technology	Responsibility
Presentation	React.js + Tailwind CSS + Recharts	Renders UI, manages local state, calls backend via Axios
Application	Node.js + Express.js	Routes requests, executes business logic, calls services
Data	MongoDB + Mongoose	Persists user data, skill profiles, interview results, roadmap progress
External	OpenAI API (gpt-3.5-turbo)	Interview answer evaluation only — isolated to one service file

Data flow: User action in React → Axios call to Express route → Auth middleware validates JWT → Controller calls service → Service reads/writes MongoDB and/or calls OpenAI → JSON response returned → React updates UI.

3. Folder Structure

Backend — /backend

Path	Contents
server.js	Entry point — Express app initialisation, middleware registration, route mounting, DB connection
config/db.js	MongoDB connection using Mongoose; async with error handling
routes/	authRoutes.js, resumeRoutes.js, quizRoutes.js, companyRoutes.js, interviewRoutes.js, roadmapRoutes.js, dashboardRoutes.js
controllers/	One controller per route file; receives req/res, calls services, sends response
services/	resumeService.js, quizService.js, scoringService.js, interviewService.js, openaiService.js, readinessService.js, roadmapService.js
models/	User.js, SkillProfile.js, InterviewResult.js, RoadmapProgress.js
middleware/	authMiddleware.js (JWT), errorMiddleware.js (global error handler), uploadMiddleware.js (Multer config)
data/	questions.json, companies.json, resources.json, interview_questions.json — static curated files
uploads/	Temporary storage for uploaded files — cleared per request

Frontend — /frontend/src

Path	Contents
pages/	Login.jsx, Register.jsx, Dashboard.jsx, ResumeUpload.jsx, Quiz.jsx, CompanyMatch.jsx, CompanyDetail.jsx, Interview.jsx, Roadmap.jsx
components/	Navbar.jsx, ScoreCard.jsx, SkillCard.jsx, QuestionCard.jsx, FeedbackCard.jsx, ChartComponent.jsx, CompanyBar.jsx,

	RoadmapWeek.jsx
services/	api.js (Axios instance with JWT header), authService.js, resumeService.js, quizService.js, companyService.js, interviewService.js, roadmapService.js
context/	AuthContext.jsx — stores user object and token; wraps the entire app
utils/	constants.js (skill list, role options), helpers.js (score formatting, colour logic)
App.js	Route definitions with React Router; protected route wrapper

Module 1 — Authentication

Responsibility

Manages user registration, login, JWT issuance, and profile updates. All other modules depend on authentication being complete before they can operate.

Files

File	Role
routes/authRoutes.js	Defines: POST /register, POST /login, GET /profile, PUT /profile
controllers/authController.js	register(), login(), getProfile(), updateProfile()
models/User.js	Schema: name, email (unique), password (hashed), targetRole, createdAt
middleware/authMiddleware.js	Extracts JWT from header, verifies with jsonwebtoken, attaches req.user

Key Logic

Registration: validate inputs → check email uniqueness → hash password with bcrypt (10 rounds) → save user → issue JWT. Login: find user by email → bcrypt.compare() → issue JWT on match → return user object without password field. JWT payload contains { userId, email, targetRole } — no sensitive data.

Inter-module dependencies

Provides: req.user (userId, targetRole) to every protected module. Consumed by: all other modules for userId filtering.

Module 2 — Resume Analyzer

Responsibility

Accepts a PDF resume, extracts raw text, detects which of 30 skills are present, and identifies which skills are missing for the user's target role. Does not persist the file.

Files

File	Role
routes/resumeRoutes.js	POST /upload
controllers/resumeController.js	Handles file upload, calls resumeService, returns results
middleware/uploadMiddleware.js	Multer config: memory storage (not disk), PDF only, 5 MB limit
services/resumeService.js	extractText(buffer), detectSkills(text, targetRole)

Key Logic

extractText(): calls pdf-parse with the file buffer from Multer memory storage. Returns raw string. File never written to disk. detectSkills(): lowercases the extracted text; iterates the 30-skill master list using includes() or regex matching; returns matched array and computes missing skills by comparing against the role's required skills from companies.json.

Inter-module dependencies

Detected skills → triggers Quiz module (user prompted per skill). Detected skills → fed into Scoring module for initial company match calculation.

Module 3 — Skill Depth Quiz Engine

Responsibility

Serves randomised MCQs per skill, scores submissions, stores depth results per user per skill, and triggers downstream recalculations after every quiz completion.

Files

File	Role
routes/quizRoutes.js	GET /quiz/:skill, POST /quiz/:skill/submit, GET /quiz/scores
controllers/quizController.js	getQuestions(), submitQuiz(), getScores()
models/SkillProfile.js	userId, skill, depthScore, depthLevel, quizAttempts, lastAttempt
services/quizService.js	loadQuestions(skill), scoreAnswers(answers, correctAnswers), classifyDepth(score)
data/questions.json	100 questions: 10 skills × 10 MCQs each, with correctAnswer field

Key Logic

getQuestions(): loads 10 questions for the skill from questions.json, shuffles using Fisher-Yates, strips the correctAnswer field before sending to client. submitQuiz(): reloads correct answers server-side, compares submitted answers, computes depthScore = (correct/10) × 100, classifies depth level, upserts SkillProfile document. After save, calls scoringService.recalculateAllMatches(userId) and readinessService.recalculate(userId).

Inter-module dependencies

Writes to: SkillProfile collection. Triggers: Company Match recalculation (scoringService), Readiness Score recalculation (readinessService).

Module 4 — Company Match Engine

Responsibility

Calculates a weighted match percentage between a student's current skill depth scores and each of 25 company profiles. Produces gap analysis showing which skills to improve first.

Files

File	Role
routes/companyRoutes.js	GET /companies, GET /companies/:id, GET /companies/:id/gap
controllers/companyController.js	getAllCompanies(), getCompany(), getGapAnalysis()
services/scoringService.js	calculateMatch(studentSkills, companyProfile), calculateGaps(studentSkills, companyProfile), recalculateAllMatches(userId)
data/companies.json	25 company profiles: id, name, sector, requiredSkills[{skill, weight, minimumDepth}]

Key Logic — Match Calculation

For each required skill in a company profile: fetch student's depthScore for that skill (0 if not taken). Compute contribution = min(depthScore / minimumDepth, 1.0) × weight. Sum all contributions and divide by total weight to get matchPercent. A student who exceeds the minimum depth for a skill gets full credit for that skill — not bonus credit.

Key Logic — Gap Analysis

For each skill where contribution < weight: compute gap = weight - contribution. Sort gaps descending by gap value. Return top 3. For each gap, compute: 'Fixing this skill would move your score from X% to Y%' by simulating a perfect depth score for that skill only.

Inter-module dependencies

Reads from: SkillProfile (student depths), companies.json (requirements). Called by: Quiz module (after every quiz submission), Interview module (after keyword feedback updates skill flags).

Module 5 — AI Mock Interview

Responsibility

Serves role-specific interview questions, sends student answers to OpenAI for dual-dimension evaluation, stores results, and feeds missed keywords back to the skill profile system.

Files

File	Role
routes/interviewRoutes.js	GET /interview/questions/:role, POST /interview/evaluate, GET /interview/history
controllers/interviewController.js	getQuestions(), evaluateAnswer(), getHistory()
models/InterviewResult.js	userId, role, question, answer, technicalScore, communicationScore, feedback fields, missedKeywords[], date
services/interviewService.js	loadQuestions(role), buildPrompt(question, answer, role)
services/openaiService.js	evaluate(prompt) — wraps OpenAI API call, parses JSON response, handles timeout and errors
data/interview_questions.json	5 questions × 10 roles = 50 questions with topic tags

Key Logic — OpenAI Prompt

The prompt instructs the model to act as a senior technical interviewer at an Indian IT company. It specifies the role context, the question, and the student's answer. It requires a strict JSON response with seven fields: technicalScore, communicationScore, technicalFeedback, communicationFeedback, missedPoint, improvedAnswer, keywordsMissed. The response is

parsed with JSON.parse(); if parsing fails (model returned non-JSON), the system falls back to storing raw text and returning a 503.

Keyword Feedback Loop

After storing the InterviewResult, keywordsMissed is processed: each keyword is mapped to its parent skill; the skill's SkillProfile document receives a 'weakFlag' update. This means a student who scores 70% on the MongoDB quiz but consistently misses 'aggregation pipeline' in interview answers will have MongoDB flagged as a weak area despite the adequate quiz score. This cross-module signal makes the roadmap more accurate.

Inter-module dependencies

Reads from: interview_questions.json. Calls: OpenAI API (openaiService). Writes to: InterviewResult, SkillProfile (weakFlags). Triggers: readinessService.recalculate(userId).

Module 6 — Placement Roadmap

Responsibility

Generates a week-by-week preparation plan ordered by gap impact for a target company. AI determines ordering; actual resources come from a static curated library. Tracks completion and feeds progress into the Readiness Score.

Files

File	Role
routes/roadmapRoutes.js	POST /roadmap/generate, GET /roadmap, PUT /roadmap/:week/complete
controllers/roadmapController.js	generateRoadmap(), getRoadmap(), markComplete()
models/RoadmapProgress.js	userId, week, skill, level, completed, completedAt
services/roadmapService.js	buildAIPrompt(gaps, weeksLeft), fillResources(orderedSkills), saveRoadmap(userId, weeks)
data/resources.json	10 skills × 2 levels × 4 resources = 80 entries: video, article, practice, task

Key Logic — Two-Step Generation

Step 1 (AI): Send gap analysis data to OpenAI. Prompt asks only for an ordered JSON array of { skill, level, reason } objects — not resource links. This costs minimal tokens and produces reliable, parseable output. Step 2 (Local): For each skill-level pair in the AI's ordered array, look up resources.json and attach the four resource links and task. Combine into the final roadmap array and save to RoadmapProgress collection, one document per week.

Inter-module dependencies

Reads from: scoringService (gaps), SkillProfile (current depths), resources.json. Writes to: RoadmapProgress. Triggers: readinessService.recalculate(userId) on markComplete().

Module 7 — Dashboard and Readiness Score

Responsibility

Aggregates data from all other modules into a single dashboard API response. Maintains and recalculates the composite Readiness Score whenever any module triggers it.

Files

File	Role
routes/dashboardRoutes.js	GET /dashboard/metrics
controllers/dashboardController.js	getMetrics() — fetches and assembles all dashboard data
services/readinessService.js	recalculate(userId) — reads all four inputs, computes and stores score

Readiness Score Formula

Component	Weight	Source	How Computed
Average depth score	35%	SkillProfile collection	Mean of all depthScore values for this user
Average interview score	30%	InterviewResult collection	Mean of (technicalScore + communicationScore) / 2 across all sessions, normalised to 100
Roadmap completion	20%	RoadmapProgress collection	(completed weeks / total weeks) × 100
Best company match	15%	scoringService	Highest matchPercent across all 25 companies for this user

Trigger Points

readinessService.recalculate(userId) is called by: quizController (after quiz submission), interviewController (after evaluation stored), roadmapController (after week marked complete). The score is stored as a timestamped entry in a ReadinessHistory sub-document on the User model, enabling the line chart on the dashboard.

Inter-module dependencies

Reads from: SkillProfile, InterviewResult, RoadmapProgress, scoringService. Called by: Quiz, Interview, Roadmap modules. Provides: the single source of truth for student progress.

---

8. Module Dependency Summary

Module	Reads From	Writes To	Triggers
Auth	Users collection	Users collection	Nothing — foundational
Resume Analyzer	companies.json (role requirements)	Nothing persisted	Quiz module (per detected skill)
Quiz Engine	questions.json, SkillProfile	SkillProfile	Company Match, Readiness Score
Company Match	SkillProfile, companies.json	Nothing persisted (computed on-demand)	Nothing
AI Interview	interview_questions.json, OpenAI API	InterviewResult, SkillProfile (weakFlags)	Readiness Score
Roadmap	scoringService, resources.json	RoadmapProgress	Readiness Score (on completion)
Dashboard	SkillProfile, InterviewResult, RoadmapProgress, scoringService	ReadinessHistory (on User)	Nothing